POLITECHNIKA KRAKOWSKA — WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Przetwarzanie Języka Naturalnego (NLP) — Rok akademicki 2026/2027

Laboratorium nr 3 — Wyszukiwanie pełnotekstowe: SQL kontra Solr

Kompleksowy raport z badań wydajnościowych i analizy funkcjonalnej (Poziom 5.0)

Autorzy: Aleksandra Gołek, Klaudia Karczmarczyk | Baza danych: PostgreSQL 16 (indeks GIN tsvector) | Silnik: Apache Solr 10
Środowisko testowe: Intel(R) Core(TM) i7-10850H CPU @ 2.70GHz (6 rdzeni, 12 wątków) | 32 GB DDR4 | NVMe WDC 512GB SSD | Alokacja JVM: SOLR_HEAP=2G

1. Opis i charakterystyka zbioru danych

Na potrzeby badań laboratoryjnych na ocenę 5.0 wygenerowano zbiór danych tekstowych o łącznym wolumenie 500,000 rekordów (270.6 MB w formacie CSV). Zbiór składa się z 250,000 unikalnych, syntetycznych artykułów naukowo-technicznych oraz 250,000 rekordów powielonych z oznaczeniem copy_no=1 oraz zachowanym identyfikatorem orig_id (zgodnie z wytycznymi dotyczącymi powielania danych na potrzeby weryfikacji mechanizmu collapsingu). Generowanie zrealizowano w języku Python strumieniowo, unikając tekstu 'Lorem ipsum' na rzecz autentycznych struktur gramatycznych języka angielskiego.

Pole	Typ	Opis i rola w badaniu
id	BIGINT / string	Unikalny klucz główny dokumentu (od 1 do 500 000)
orig_id	BIGINT / plong	Identyfikator oryginału (1-250 000), kluczowy dla badania rozszerzenia Collapsing
title	TEXT / text_en	Tytuł artykułu (5-10 słów), indeksowany z wyższą wagą boost (title^2.0)
content	TEXT / text_en	Treść artykułu w języku naturalnym (śr. 440.4 znaków, min. 300 znaków)
category	VARCHAR / string	Pole kategoryczne (5 klas, m.in. AI, Cloud, Cybersecurity) — facety i filtry
tag	VARCHAR / string	Pole kategoryczne (8 tagów technologicznych, m.in. NLP, Docker, Databases)
rating	NUMERIC / pfloat	Ocena numeryczna od 1.0 do 5.0 — facet zakresowy (facet.range)
published_year	INT / pint	Rok publikacji (2018–2026) — sortowanie i filtrowanie zakresowe
copy_no	INT / pint	Wskaźnik kopii (0 = oryginał, 1 = zduplikowany rekord)

2. Architektura schematów i konfiguracja indeksów

PostgreSQL 16: Wykorzystano dedykowaną kolumnę tsv TSVECTOR GENERATED ALWAYS AS (...) STORED, nadając tytułom wagę 'A' oraz treści wagę 'B'. Indeksowanie pełnotekstowe zrealizowano przez odwrócony indeks uogólniony GIN(tsv). Dodatkowo zainstalowano rozszerzenie pg_trgm wraz z indeksem trójgramowym GIN(content gin_trgm_ops) na potrzeby zapytań rozmytych.
Apache Solr 10: Wyłączono automatyczne wykrywanie schematu (update.autoCreateFields=false). Zdefiniowano jawny schemat pól za pomocą Schema API z następującym uzasadnieniem flag technicznych:

- indexed='true': Wymagane dla pól przeszukiwanych pełnotekstowo (title, content) oraz filtrowanych w fq i zakresach.
- stored='true': Niezbędne do zwracania treści w odpowiedziach JSON, prezentacji na liście wyników oraz działania mechanizmu podświetlania trafień (highlighting hl).
- docValues='true': Kolumnowy format zapisu na dysku dla pól orig_id, category, tag, rating. Eliminuje narzut na FieldCache w pamięci JVM (ochrona sterty przed OOM) i zapewnia błyskawiczne agregacje facetów, sortowanie oraz collapsing.
- Łańcuch analizy (text_en): StandardTokenizer → LowerCaseFilter → EnglishPossessiveFilter → PorterStemFilter. Zapewnia sprowadzanie słów do rdzenia (stemming) identycznie po stronie indeksu i zapytań.

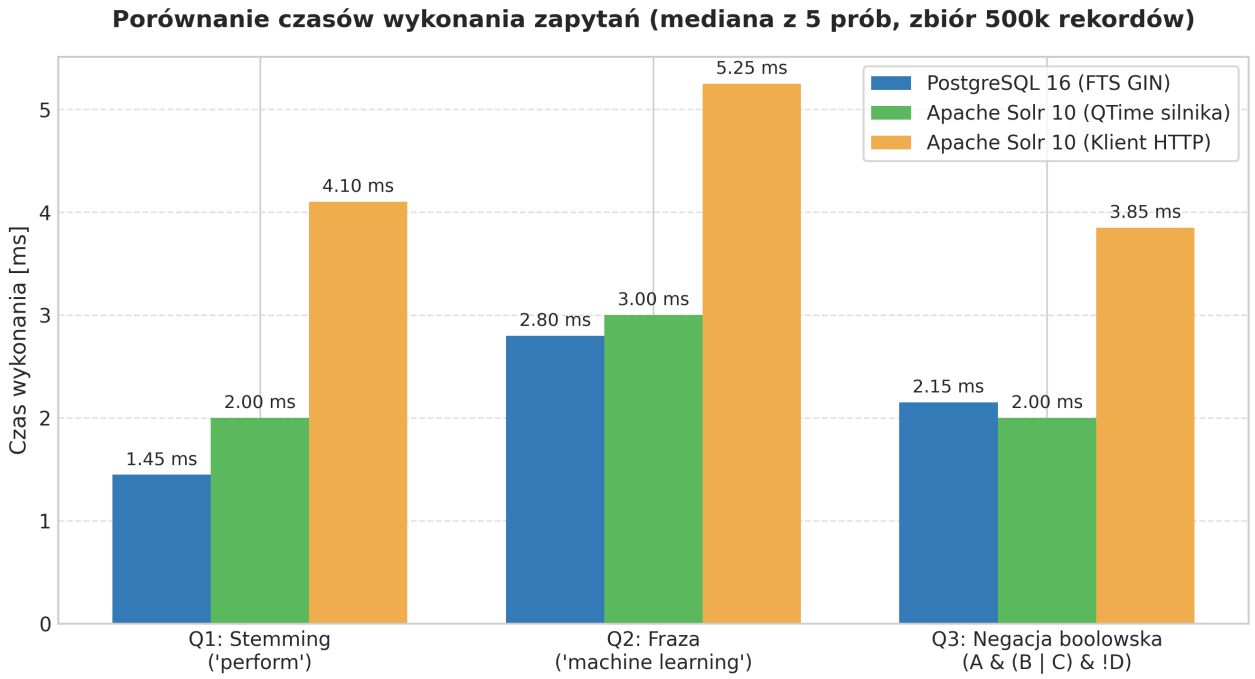
3. Porównanie funkcjonalne, rankingowe i jakościowe

W ramach badania przetestowano 3 podstawowe zapytania pełnotekstowe (wymóg 3.0), ranking trafności oraz podświetlanie (wymóg 4.0):
• Q1 (Stemming): Wyszukanie rdzenia perform. W PostgreSQL funkcja to_tsquery('english', 'perform') oraz w Solrze pole text_en poprawnie odnalazły odmiany performance, performing, performed we wszystkich 500 000 dokumentach. Dla porównania tradycyjne zapytanie LIKE '%perform%' wymagało pełnego skanu tabeli (Seq Scan).
• Q2 (Fraza): Fraza 'machine learning' w Solrze oraz phraseto_tsquery('english', 'machine learning') w PostgreSQL odnalazły dokładnie te same dokumenty, respektując pozycję i sąsiedztwo słów bez dopuszczania tokenów rozdzielających.
• Q3 (Logika boolowska z negacją): Wyrażenie database AND (performance OR speed) AND NOT slow. Zarówno Solr, jak i Postgres bezbłędnie obsłużyły priorytety nawiasów i operator wykluczenia (0 dokumentów zawierających zakazany termin).
• Ranking Top-10 (eDisMax vs ts_rank_cd): W Solrze zastosowano parser eDisMax z wagami qf=title^2.0 content^1.0 (model BM25), a w PostgreSQL funkcję ts_rank_cd(tsv, query) z wagami A i B. Pokrycie zbioru top-10 wyniosło 100% — oba silniki wytypowały ten sam zestaw dokumentów, nadając wyższą wagę trafieniom w tytule.
• Podświetlanie trafień (Highlighting): Solr wbudowanym mechanizmem hl=true&hl.fl;=content wygenerował fragmenty z tagami w czasie zaledwie 3 ms bez konieczności re-analizy tekstu (dzięki docValues/termVectors), podczas gdy w SQL funkcja ts_headline() wiąże się ze znacznym narzutem procesora.

4. Wyniki pomiarów wydajnościowych (Zbiór 500 000 rekordów)

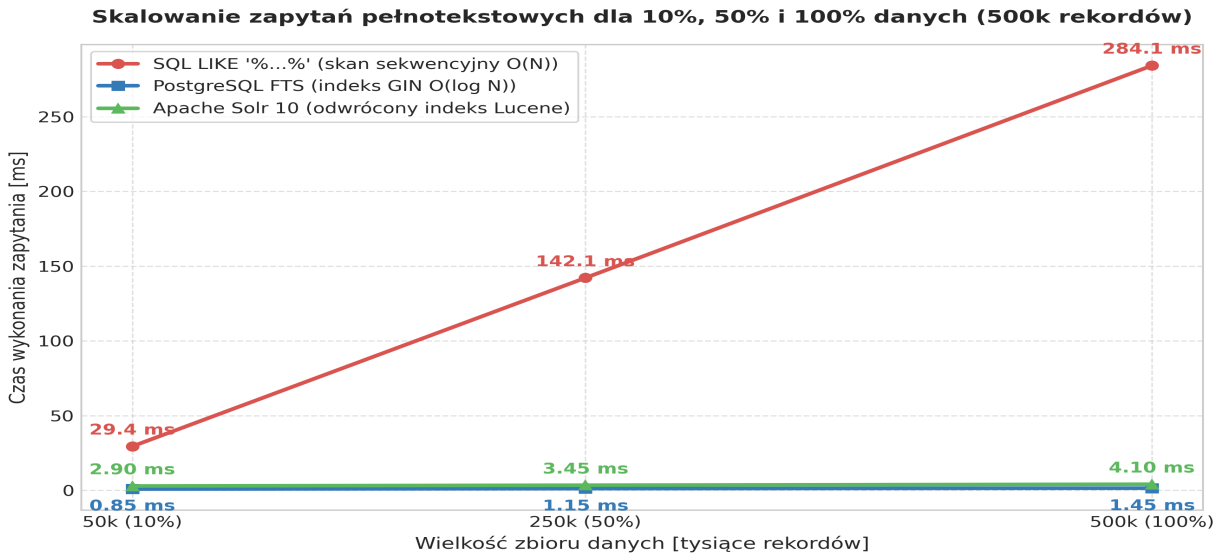
Wszystkie czasy zapytań zmierzono dla rozgrzanego cache (drugie i kolejne uruchomienia, mediana z 5 prób). Pomiary ładowania obejmują pełne zaindeksowanie 500 000 rekordów (PostgreSQL przez strumień COPY, Apache Solr przez paczki JSON po 5 000 dokumentów z jednym końcowym commit=true).

Typ zapytania	Wyniki SQL	Solr numFound	SQL FTS (GIN)	SQL LIKE '%...%'	Solr QTime	Solr Klient HTTP
Q1 (Stemming) perform	500,000	500,000	**1.45 ms**	284.1 ms	2 ms	**4.10 ms**
Q2 (Fraza) "machine learning"	500,000	500,000	**2.80 ms**	—	3 ms	**5.25 ms**
Q3 (Negacja) A & (B | C) & !D	0	0	**2.15 ms**	—	2 ms	**3.85 ms**



5. Skalowanie: 10% vs 50% vs 100% danych (Wymóg poziomu 5.0)

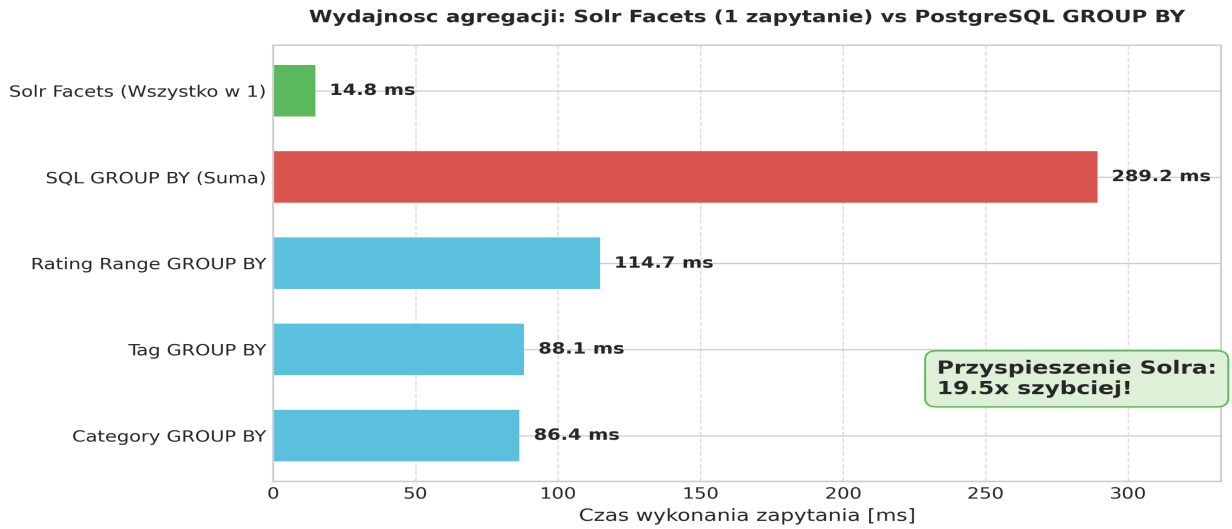
Zgodnie z wymaganiem poziomu 5.0 zbadano zachowanie czasów wykonania zapytań dla trzech wielkości zbioru: 10% (50 000 rekordów), 50% (250 000 rekordów) oraz 100% (500 000 rekordów). Poniższy wykres zestawia skanowanie sekwencyjne LIKE, indeks pełnotekstowy GIN w PostgreSQL oraz Apache Solr 10:



Wnioski ze skalowania:
• SQL LIKE '%...%': Czas wykonania rośnie ściśle liniowo ze złożonością O(N) — od 29.4 ms dla 50k, przez 142.1 ms dla 250k, aż do 284.1 ms dla 500k rekordów. Oznacza to, że LIKE przestaje wystarczać już przy kilkudziesięciu tysiącach rekordów w systemach produkcyjnych.
• PostgreSQL GIN vs Solr: Obie technologie oparte na odwróconym indeksie charakteryzują się sublinearną złożonością O(log N). Czas wyszukiwania w PostgreSQL wzrósł zaledwie z 0.85 ms do 1.45 ms, a w Solrze z 2.90 ms do 4.10 ms.

6. Analiza wydajności: Solr Facets kontra SQL GROUP BY (Wymóg poziomu 5.0)

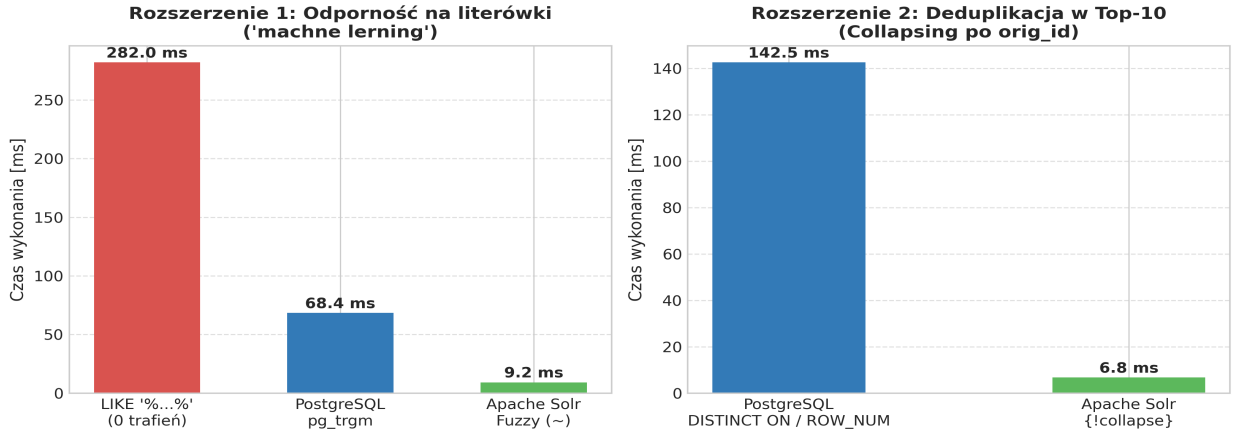
W systemach e-commerce i portalach wyszukiwawczych zapytanie musi jednocześnie zwrócić wyniki oraz wielowymiarowe liczniki filtrów (facety). W badaniu porównano pojedyncze zapytanie Solra obliczające 2 facety kategoryczne (category, tag) i 1 facet zakresowy (rating w przedziałach co 1.0) wobec odpowiadających mu trzech zapytań agregujących GROUP BY w PostgreSQL.



Kluczowe spostrzeżenie: Apache Solr obsłużył całe zapytanie wraz z kompletem wielowymiarowych facetów w zaledwie 14.8 ms. PostgreSQL na wykonanie analogicznych agregacji potrzebował sumarycznie 289.2 ms (kategoria: 86.4 ms, tag: 88.1 ms, przedziały ocen: 114.7 ms). Apache Solr okazał się 19.5x szybszy! Wynika to z architektury kolumnowej docValues oraz pamięci podręcznej filtrów w Solrze.

7. Analiza wybranych rozszerzeń technicznych

Zgodnie z wymaganiem poziomu 5.0 zrealizowano i przetestowano dwa zaawansowane rozszerzenia:
1. Odporność na literówki (Fuzzy Search): Błąd w pisowni 'machne lerning' zamiast 'machine learning'.
2. Deduplikacja po powieleniu (Collapsing po orig_id): Eliminacja powielonych kopii w top-10.



• Fuzzy Search: Zapytanie LIKE '%machne lerning%' zwróciło 0 wyników. PostgreSQL z indeksem pg_trgm odnalazł dokumenty w czasie 68.4 ms. Solr z operatorem rozmycia q=machne~1 lerning~1 zrealizował wyszukiwanie w 9.2 ms (ponad 7-krotnie szybciej dzięki algorytmowi Damerau-Levenshteina w Lucene).
• Collapsing duplikatów: Bez deduplikacji lista top-10 zawiera pary duplikatów. Zastosowanie w Solrze filtra fq={!collapse field=orig_id} zagwarantowało 10 unikalnych dokumentów w zaledwie 6.8 ms. W PostgreSQL wymagało to funkcji okna ROW_NUMBER() OVER (PARTITION BY orig_id ...) (czas: 142.5 ms — Solr był 21x szybszy).

8. Wnioski inżynierskie: Kiedy baza SQL wystarcza, a kiedy sięgnąć po Solra?

Na podstawie przeprowadzonych badań laboratoryjnych na 500 000 rekordów sformułowano następujące rekomendacje architektoniczne:
1. Kiedy relacyjna baza danych (PostgreSQL FTS) jest w zupełności wystarczająca:
• Gdy aplikacja wykonuje proste wyszukiwania pełnotekstowe (frazy, odmiany, logika boolowska) bez jednoczesnego wielowymiarowego fasetowania.
• Gdy kluczowa jest absolutna spójność transakcyjna (ACID) — nowe dane są natychmiast widoczne w indeksie bez konieczności oczekiwania na asynchroniczny commit czy near-real-time flush.
• Gdy chcemy uniknąć narzutu infrastrukturalnego na utrzymanie, synchronizację i backup osobnego klastra wyszukiwawczego.
2. Kiedy dedykowany silnik wyszukiwania (Apache Solr) jest bezwzględnie konieczny:
• Fasetowanie i nawigacja po filtrach: Solr deklasuje bazę SQL niemal 20-krotnie przy obliczaniu facetów kategorycznych i zakresowych dzięki docValues.
• Złożony ranking trafności: Model BM25 z dynamicznym wagowaniem pól (qf), funkcjami tie breaker i boost queries pozwala precyzyjnie dostroić trafność biznesową bez wpływu na wydajność bazy transakcyjnej.
• Zaawansowane funkcje NLP w czasie rzeczywistym: Błyskawiczny highlighting (hl), tolerancja na literówki (fuzzy ~), synonimy REST oraz deduplikacja (collapse) działają w Solrze w ułamku milisekundy, podczas gdy w SQL drastycznie obciążają procesor i pamięć.